LABORATORY ASSIGNMENT REPORT

PAPER: COMPUTER SCIENCE & APPLICATION

DEPARTMENT OF COMPUTER SCIENCE

SALTBROOK MIRZA SENIOR SECONDARY SCHOOL

LABORATORY ASSIGNMENT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF HS CERTIFICATE
UNDER
ASSAM STATE SCHOOL EDUCATION BOARD (ASSEB)
2026-27

Submitted by

Student name : **Gaurav Kalita**
Roll : .....No: .....
Reg. No: .....
HS Final Year, 2026-27
SALTBROOK MIRZA SENIOR SECONDARY SCHOOL

HS FINAL YEAR 2026-27

DEPARTMENT OF COMPUTER SCIENCE

SALTBROOK MIRZA SENIOR SECONDARY SCHOOL

MIRZA, KAMRUP-781125

Date:.....

TO WHOM IT MAY CONCERN

This is to certify that **Gaurav Kalita of H.S. Final Year under Assam State School Education Board (ASSEB), SALTBROOK MIRZA SENIOR SECONDARY SCHOOL,** *has completed all the laboratory assignments for the Subject Paper:* **"COMPUTER SCIENCE AND APPLICATION", Department of Computer Science,** *with his/her own effort within the stipulated time.*

We wish him/her all success in life.

Signature of
External Examiner

Department of Computer Science
SALTBROOK MIRZA SENIOR SECONDARY SCHOOL

INDEX

INDEX

ID	Name	Role	Salary	Joining_date	Company
1	Rahul Sharma	Developer	50000	2024-11-01	Skillarger
2	Anjali Mehta	Developer	45000	2025-10-15	Skillarger
3	Amitabh Sinha	Manager	70000	2025-09-20	Skillarger
4	Priya Singh	HR	40000	2026-07-10	Skillarger
5	Ravi Patel	Analyst	55000	2026-08-05	Skillarger

- **i)** Write a SQL command to create the columns of above table and name it Employees.
- **ii)** Write a SQL command to insert the necessary records to create above table.
- **iii)** Write a SQL command to retrieve all the records.
- **iv)** Write a SQL command to retrieve Name and Role columns from the table.
- **v)** Write a SQL command to retrieve records of Salary greater than 50000.
- **vi)** Write a SQL command to update Salary of record with ID 2 to 60000.
- **vii)** Write a SQL command to delete record with ID 4.
- **viii)** Write a SQL command to count total number of records of Employees.
- **ix)** Write a SQL command to show records in descending order of Salary.
- **x)** Write a SQL command to find the highest salary in Employees.
- **xi)** Write a SQL command to retrieve number of Employees in each role.
- **xii)** Write a SQL command to remove the table.
- **xiii)** Write a SQL command to add a new column Email to Employees table.
- **xiv)** Write a SQL command to update Email to an appropriate value.
- **xv)** Write a SQL command to retrieve ID, Name and Email from Employees.
- **xvi)** Write a SQL command to find records of Employees who joined after 1/9/2024.
- **xvii)** Write a SQL command to rename column Email to Contact_Email.
- **xviii)** Write a SQL command to add a new employee with the necessary fields.
- **xix)** Write a SQL command to show new table with updated records.
- **xx)** Write a SQL command to create a view HighEarners of Employees with Salary greater than 50000.
- **xxi)** Write a SQL command to retrieve view HighEarners.

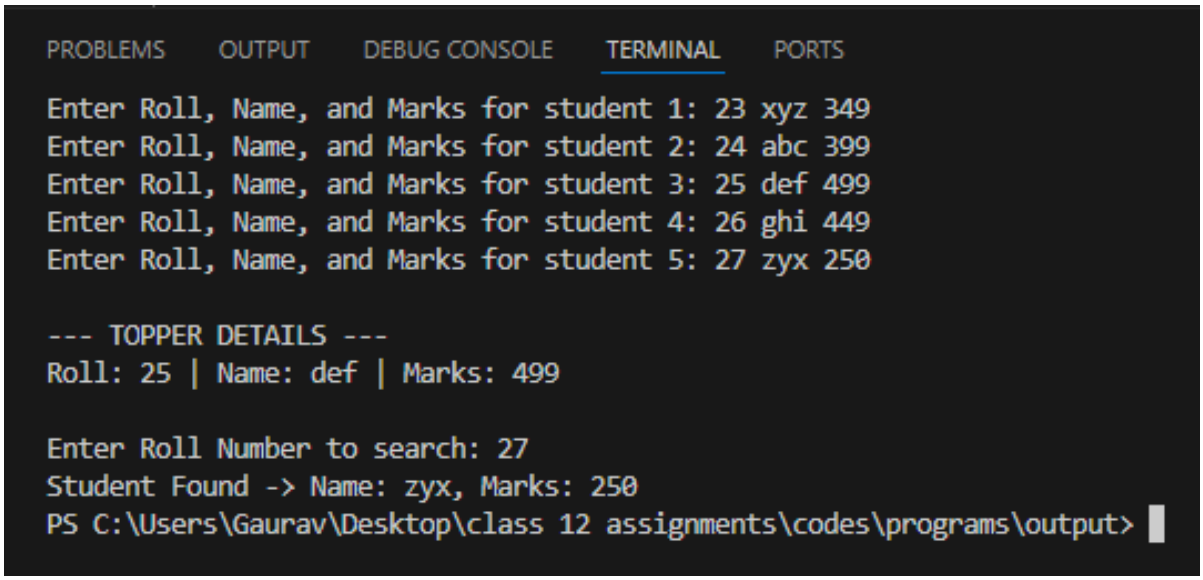
Program 1. : Program to hold details of 5 students and provide the facility of viewing details of the topper as well as of specific student by providing his/her roll number.

Soln:

```
#include <iostream>
using namespace std;
struct Student {
   int roll;
   string name;
   float marks;
};
int main() {
   Student s[5];
   int topperIndex = 0;
   for (int i = 0; i < 5; i++) {
      cout << "Enter Roll, Name, and Marks for student " << i + 1 << ": ";
      cin >> s[i].roll >> s[i].name >> s[i].marks;
      if (s[i].marks > s[topperIndex].marks) {
         topperIndex = i;
      }
   }
   cout << "\n--- TOPPER DETAILS ---\n";
   cout << "Roll: " << s[topperIndex].roll
       << " | Name: " << s[topperIndex].name
       << " | Marks: " << s[topperIndex].marks << endl;
   int searchRoll;
   cout << "\nEnter Roll Number to search: ";
   cin >> searchRoll;
   bool found = false;
   for (int i = 0; i < 5; i++) {
      if (s[i].roll == searchRoll) {
         cout << "Student Found -> Name: " << s[i].name
```

```
            << ", Marks: " << s[i].marks << endl;
            found = true;
            break;
        }
    }
    if (!found) cout << "Student with Roll Number " << searchRoll << " not found.\n";
    return 0;
}
```

Output:



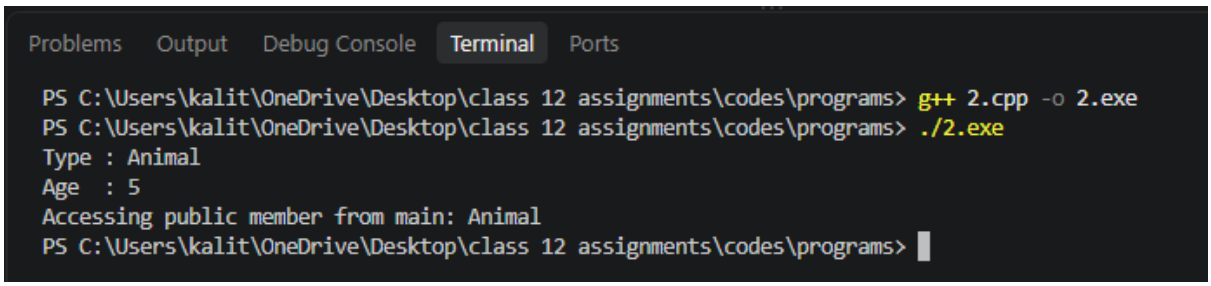
Program 2. : Program to illustrate the access control in public derivation of a class with the concept of inheritance.

Soln:

```
#include <iostream>
using namespace std;
class Animal{
public:
   string type;
protected:
   int age;
private:
   string secretCode;
public:
   Animal(){
      type = "Animal";
      age = 5;
      secretCode = "A123";
   }
};
class Dog : public Animal{
public:
   void display(){
      cout << "Type : " << type << endl;
      cout << "Age  : " << age << endl;
   }
};
int main(){
   Dog d;
   d.display();
   cout << "Accessing public member from main: " << d.type << endl;
   return 0;
```

```
}
```

Output:

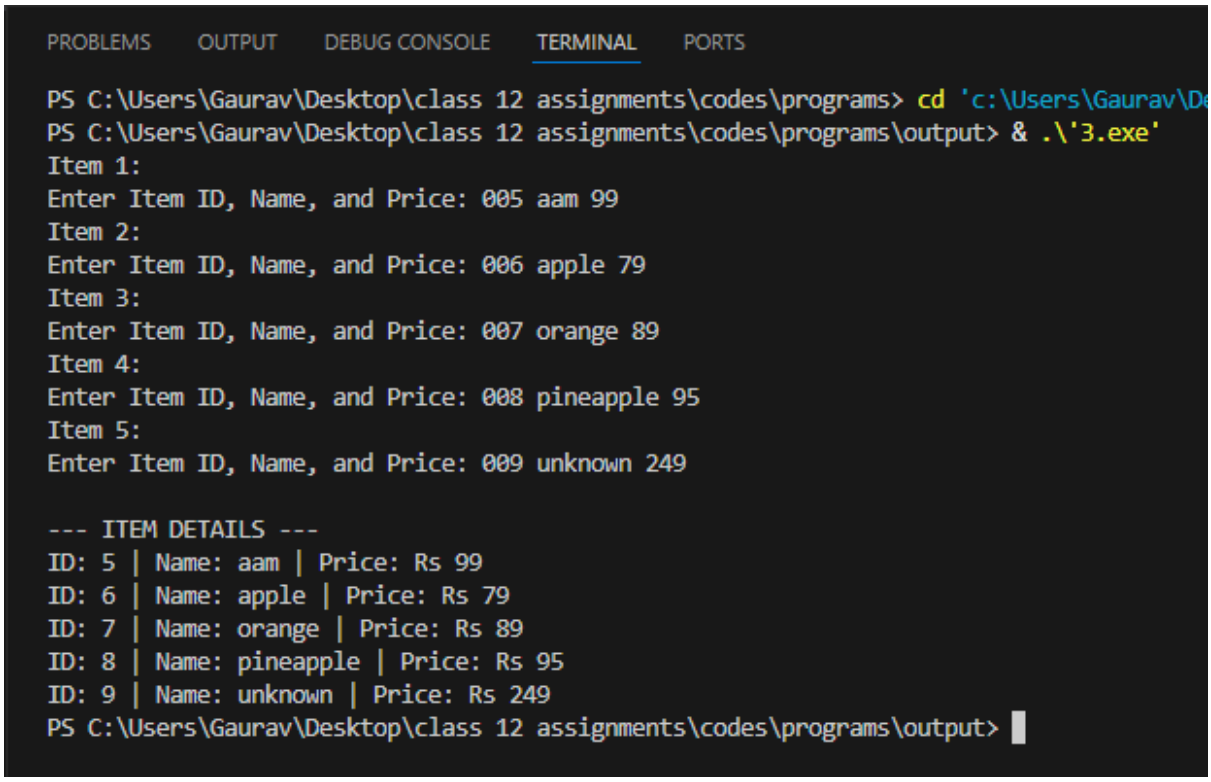

Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\kalit\OneDrive\Desktop\class 12 assignments\codes\programs> g++ 2.cpp -o 2.exe
PS C:\Users\kalit\OneDrive\Desktop\class 12 assignments\codes\programs> ./2.exe
Type : Animal
Age : 5
Accessing public member from main: Animal
PS C:\Users\kalit\OneDrive\Desktop\class 12 assignments\codes\programs>

Program 3. : Program to illustrate the use of object arrays by storing details of 5 items in an array of objects.

Soln:

```
#include <iostream>
using namespace std;
class Item {
   int id;
   string name;
   float price;
public:
   void getItem() {
      cout << "Enter Item ID, Name, and Price: ";
      cin >> id >> name >> price;
   }
   void putItem() const {
      cout << "ID: " << id << " | Name: " << name << " | Price: Rs " << price << endl;
   }
};
int main() {
   Item items[5];
   for (int i = 0; i < 5; i++) {
      cout << "Item " << i + 1 << ":\n";
      items[i].getItem();
   }
   cout << "\n--- ITEM DETAILS ---\n";
   for (int i = 0; i < 5; i++) {
      items[i].putItem();
   }
   return 0;
}
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs> cd 'c:\Users\Gaurav\D
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'3.exe'
Item 1:
Enter Item ID, Name, and Price: 005 aam 99
Item 2:
Enter Item ID, Name, and Price: 006 apple 79
Item 3:
Enter Item ID, Name, and Price: 007 orange 89
Item 4:
Enter Item ID, Name, and Price: 008 pineapple 95
Item 5:
Enter Item ID, Name, and Price: 009 unknown 249
--- ITEM DETAILS ---
ID: 5 | Name: aam | Price: Rs 99
ID: 6 | Name: apple | Price: Rs 79
ID: 7 | Name: orange | Price: Rs 89
ID: 8 | Name: pineapple | Price: Rs 95
ID: 9 | Name: unknown | Price: Rs 249
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output>

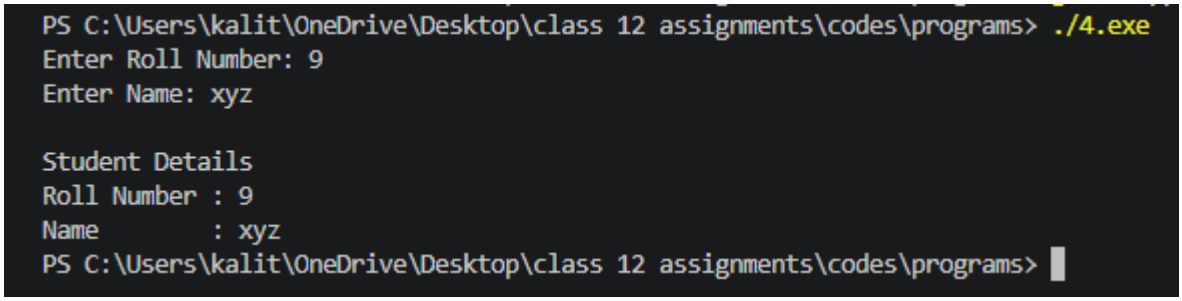
Program 4. : Program to illustrate the working of a function returning an object.

Soln:

```
#include <iostream>
using namespace std;
class Student{
private:
   int rollNo;
   string name;
public:
   void getData(){
      cout << "Enter Roll Number: ";
      cin >> rollNo;
      cout << "Enter Name: ";
      cin >> name;
   }
   void display(){
      cout << "\nStudent Details" << endl;
      cout << "Roll Number : " << rollNo << endl;
      cout << "Name       : " << name << endl;
   }
};
Student createStudent(){
   Student s;
   s.getData();
   return s;
}
int main(){
   Student s1;
   s1 = createStudent();
   s1.display();
   return 0;
```

```
}
```

Output:


PS C:\Users\kalit\OneDrive\Desktop\class 12 assignments\codes\programs> ./4.exe
Enter Roll Number: 9
Enter Name: xyz
Student Details
Roll Number : 9
Name : xyz
PS C:\Users\kalit\OneDrive\Desktop\class 12 assignments\codes\programs>

Program 5. : Program to illustrate the Call By Reference mechanism on objects.

Soln:

```
#include <iostream>
using namespace std;
class BankAccount{
private:
   string holderName;
   float balance;
public:
   void getData(){
      cout << "Enter Account Holder Name: ";
      cin >> holderName;
      cout << "Enter Balance: ";
      cin >> balance;
   }
   void display(){
      cout << "\nAccount Holder : " << holderName << endl;
      cout << "Balance       : " << balance << endl;
   }
   friend void deposit(BankAccount &, float);
};
void deposit(BankAccount &acc, float amount){
   acc.balance += amount;
}
int main(){
   BankAccount account;
   account.getData();
   cout << "\nBefore Deposit:";
   account.display();
   deposit(account, 5000);
   cout << "\nAfter Depositing Rs.5000:";
```

```
    account.display();

    return 0;

}
```

Output :

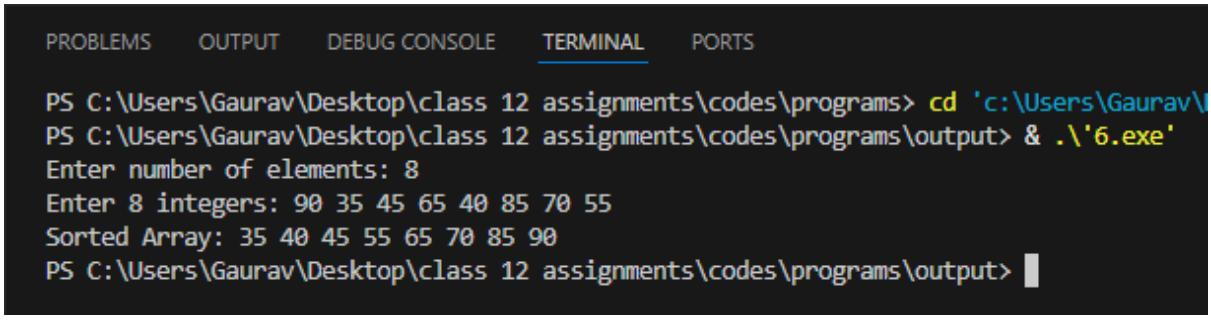
```
PS C:\Users\kalit\OneDrive\Desktop\class 12 assignments\codes\programs> ./5.exe
Enter Account Holder Name: xyz
Enter Balance: 67000

Before Deposit:
Account Holder : xyz
Balance        : 67000

After Depositing Rs.5000:
Account Holder : xyz
Balance        : 72000
PS C:\Users\kalit\OneDrive\Desktop\class 12 assignments\codes\programs>
```

Program 6. : Program to Sort elements of an array.

Soln:

```
#include <iostream>

using namespace std;

int main() {

   int n, arr[100];

   cout << "Enter number of elements: ";

   cin >> n;

   cout << "Enter " << n << " integers: ";

   for (int i = 0; i < n; i++) cin >> arr[i];

   for (int i = 0; i < n - 1; i++) {

      for (int j = 0; j < n - i - 1; j++) {

         if (arr[j] > arr[j + 1]) {

            int temp = arr[j];

            arr[j] = arr[j + 1];

            arr[j + 1] = temp;

         }

      }

   }

   cout << "Sorted Array: ";

   for (int i = 0; i < n; i++) cout << arr[i] << " ";

   cout << endl;

   return 0;

}
```

Output:

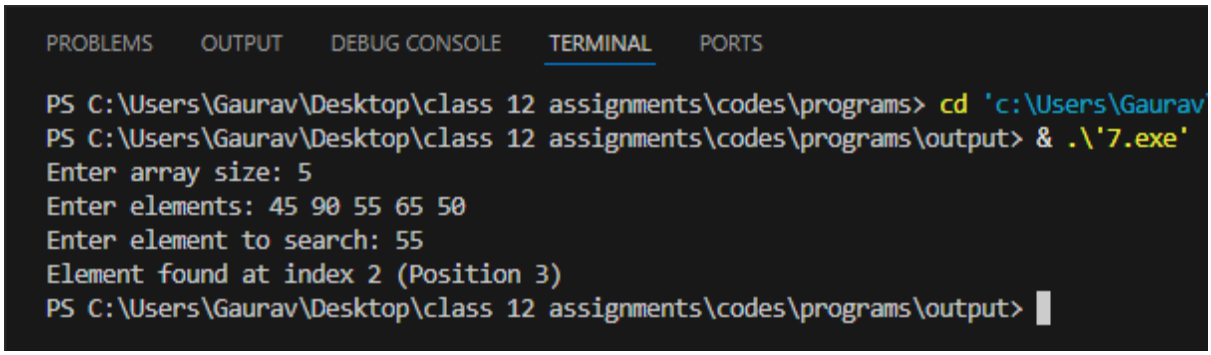


Program 7. : Program demonstrating Linear searching.

Soln:

```
#include <iostream>
using namespace std;
int main() {
   int n, key, arr[100];
   cout << "Enter array size: ";
   cin >> n;
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++) cin >> arr[i];
   cout << "Enter element to search: ";
   cin >> key;
   int pos = -1;
   for (int i = 0; i < n; i++) {
      if (arr[i] == key) {
         pos = i;
         break;
      }
   }if (pos != -1)
      cout << "Element found at index " << pos << " (Position " << pos + 1 << ")\n";
   else
      cout << "Element not found.\n";
   return 0;
}
```

Output:

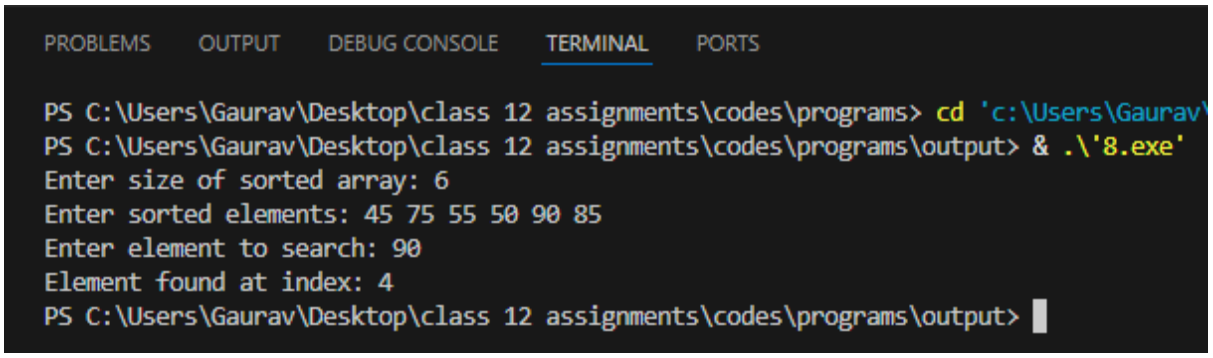


Program 8. : Program demonstrating Binary search.

Soln:

```
#include <iostream>
using namespace std;
int main() {
   int n, key, arr[100];
   cout << "Enter size of sorted array: ";
   cin >> n;
   cout << "Enter sorted elements: ";
   for (int i = 0; i < n; i++) cin >> arr[i];
   cout << "Enter element to search: ";
   cin >> key;
   int low = 0, high = n - 1, mid, index = -1;
   while (low <= high) {
      mid = low + (high - low) / 2;
      if (arr[mid] == key) {
         index = mid;
         break;
      } else if (arr[mid] < key) {
         low = mid + 1;
      } else {
         high = mid - 1;
      }
   }if (index != -1)
      cout << "Element found at index: " << index << endl;
   else
      cout << "Element not found.\n";
   return 0;
}
```

Output:

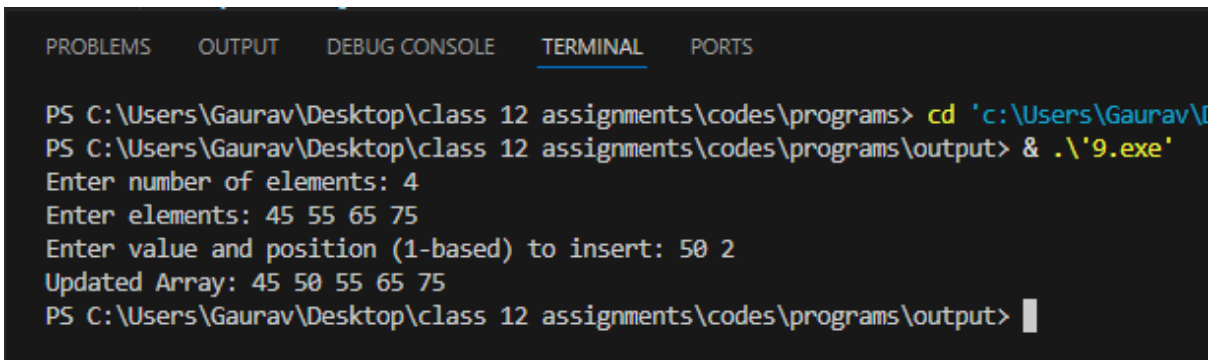

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs> cd 'c:\Users\Gaurav
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'8.exe'
Enter size of sorted array: 6
Enter sorted elements: 45 75 55 50 90 85
Enter element to search: 90
Element found at index: 4
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output>

Program 9. : Program for Inserting elements into the array.

Soln:

```
#include <iostream>
using namespace std;
int main() {
   int n, val, pos, arr[100];
   cout << "Enter number of elements: ";
   cin >> n;
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++) cin >> arr[i];
   cout << "Enter value and position (1-based) to insert: ";
   cin >> val >> pos;
   for (int i = n; i >= pos; i--) {
      arr[i] = arr[i - 1];
   }
   arr[pos - 1] = val;
   n++;
   cout << "Updated Array: ";
   for (int i = 0; i < n; i++) cout << arr[i] << " ";
   cout << endl;
   return 0;
}
```

Output:

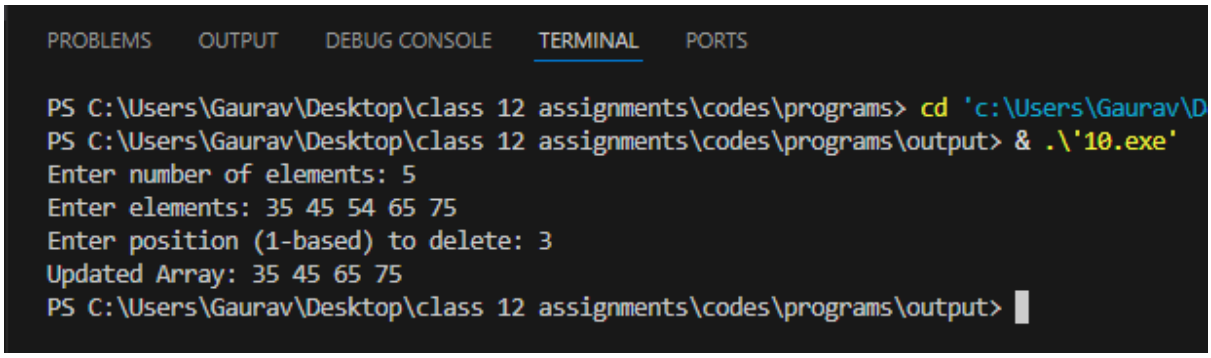


Program 10. : Program for Deleting elements on the array.

Soln:

```
#include <iostream>
using namespace std;
int main() {
   int n, pos, arr[100];
   cout << "Enter number of elements: ";
   cin >> n;
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++) cin >> arr[i];
   cout << "Enter position (1-based) to delete: ";
   cin >> pos;
   if (pos < 1 || pos > n) {
      cout << "Invalid position!\n";
   } else {
      for (int i = pos - 1; i < n - 1; i++) {
         arr[i] = arr[i + 1];
      }
      n--;
      cout << "Updated Array: ";
      for (int i = 0; i < n; i++) cout << arr[i] << " ";
      cout << endl;
   }
   return 0;
}
```

Output:

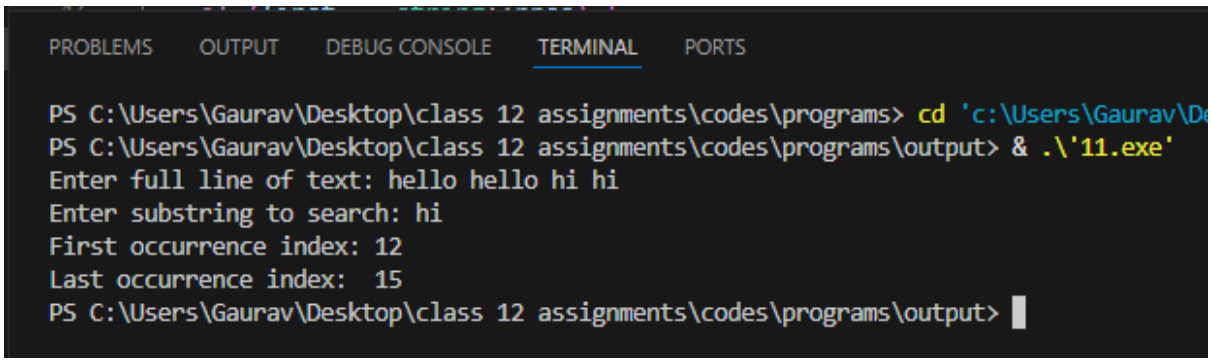

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs> cd 'c:\Users\Gaurav\D
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'10.exe'
Enter number of elements: 5
Enter elements: 35 45 54 65 75
Enter position (1-based) to delete: 3
Updated Array: 35 45 65 75
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output>

Program 11. : Write a C++ program that inputs a line of text and a substring to search for. Then it should display the position of the first and last occurrence of the word.

Soln:

```
#include <iostream>
#include <string>
using namespace std;
int main() {
   string text, sub;
   cout << "Enter full line of text: ";
   getline(cin, text);
   cout << "Enter substring to search: ";
   cin >> sub;
   size_t first = text.find(sub);
   size_t last = text.rfind(sub);
   if (first == string::npos) {
      cout << "Substring not found in the given text.\n";
   } else {
      cout << "First occurrence index: " << first << endl;
      cout << "Last occurrence index:  " << last << endl;
   }
   return 0;
}
```

Output:

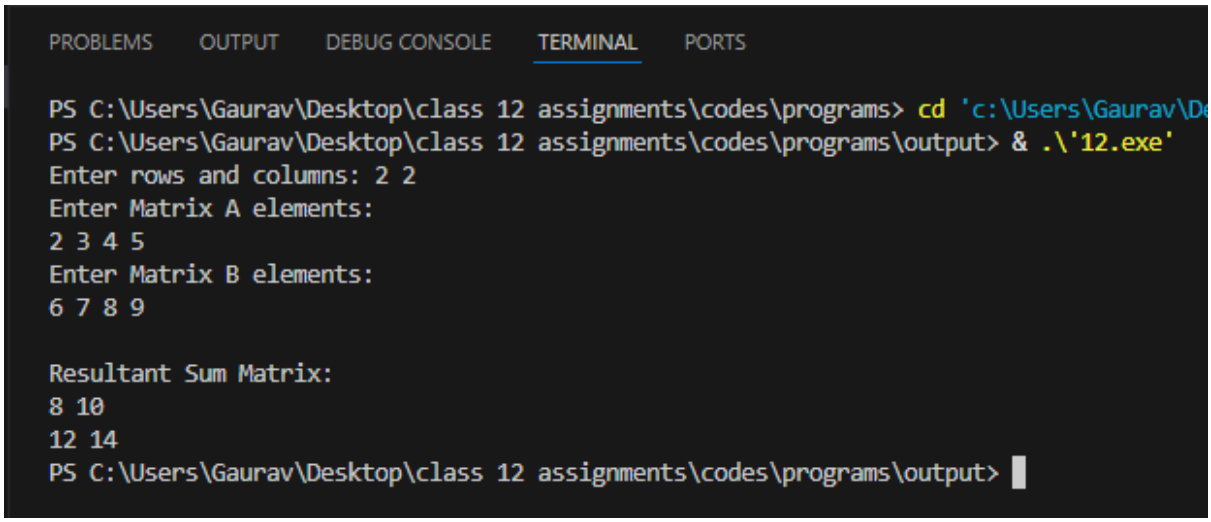


Program 12. : Program to illustrate summation of Matrices.

Soln:

```
#include <iostream>
using namespace std;
int main() {
   int r, c, a[10][10], b[10][10], sum[10][10];
   cout << "Enter rows and columns: ";
   cin >> r >> c;
   cout << "Enter Matrix A elements:\n";
   for (int i = 0; i < r; i++)
      for (int j = 0; j < c; j++)
         cin >> a[i][j];
   cout << "Enter Matrix B elements:\n";
   for (int i = 0; i < r; i++)
      for (int j = 0; j < c; j++)
         cin >> b[i][j];
   for (int i = 0; i < r; i++)
      for (int j = 0; j < c; j++)
         sum[i][j] = a[i][j] + b[i][j];
   cout << "\nResultant Sum Matrix:\n";
   for (int i = 0; i < r; i++) {
      for (int j = 0; j < c; j++) {
         cout << sum[i][j] << " ";
      }
      cout << endl;
   }
   return 0;
}
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs> cd 'c:\Users\Gaurav\De
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'12.exe'
Enter rows and columns: 2 2
Enter Matrix A elements:
2 3 4 5
Enter Matrix B elements:
6 7 8 9
Resultant Sum Matrix:
8 10
12 14
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output>

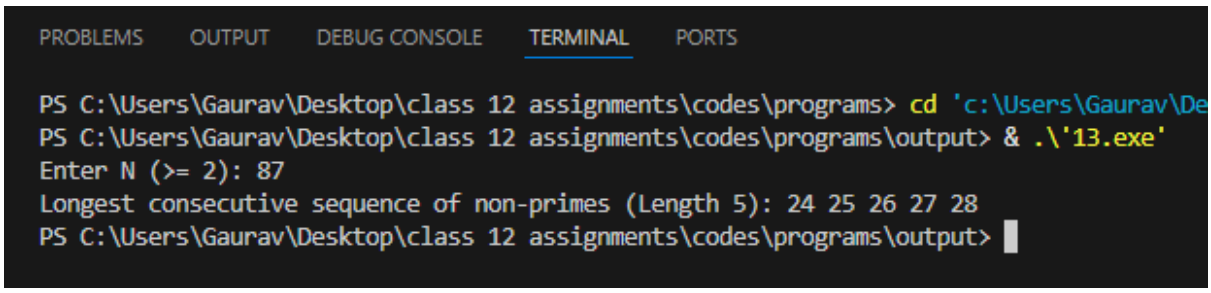
Program 13. : Write a C++ program to find the Longest consecutive sequence of integers with no primes. The program should accept a number N and print out the Largest block of integers between 2 and N with no primes.

Soln:

```
#include <iostream>
using namespace std;
bool isPrime(int n) {
   if (n < 2) return false;
   for (int i = 2; i * i <= n; i++) {
      if (n % i == 0) return false;
   }
   return true;
}
int main() {
   int n;
   cout << "Enter N (>= 2): ";
   cin >> n;
   int maxLen = 0, maxStart = 0;
   int curLen = 0, curStart = 0;
   for (int i = 2; i <= n; i++) {
      if (!isPrime(i)) {
         if (curLen == 0) curStart = i;
         curLen++;
         if (curLen > maxLen) {
            maxLen = curLen;
            maxStart = curStart;
         }} else {
         curLen = 0;
      }
   }
   if (maxLen > 0) {
```

```
        cout << "Longest consecutive sequence of non-primes (Length " << maxLen << "): ";
        for (int i = maxStart; i < maxStart + maxLen; i++) {
            cout << i << " ";
        }
        cout << endl;
    } else {
        cout << "No composite sequence found between 2 and " << n << ".\n";
    }
    return 0;
}
```

Output:



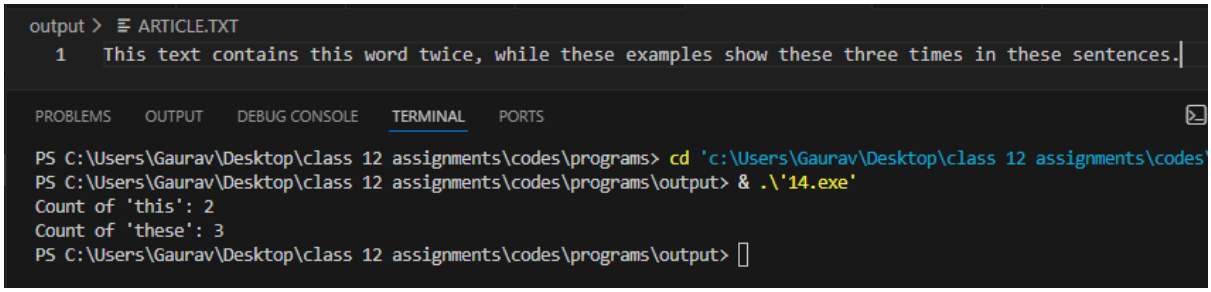
Program 14. : Write a function in C++ to count the words "this" and "these" present in a text file "ARTICLE.TXT".

Soln:

```
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>
using namespace std;
void countWords() {
   ifstream file("ARTICLE.TXT");
   if (!file) {
      cout << "Error: Cannot open ARTICLE.TXT\n";
      return;
   }
   string word;
   int countThis = 0, countThese = 0;
   while (file >> word) {
      string cleanWord = "";
      for (char ch : word) {
         if (isalpha(ch)) cleanWord += tolower(ch);
      }
      if (cleanWord == "this") countThis++;
      if (cleanWord == "these") countThese++;
   }
   file.close();
   cout << "Count of 'this': " << countThis << endl;
   cout << "Count of 'these': " << countThese << endl;
}
int main() {
   countWords();
```

```
    return 0;
}
```

Output:

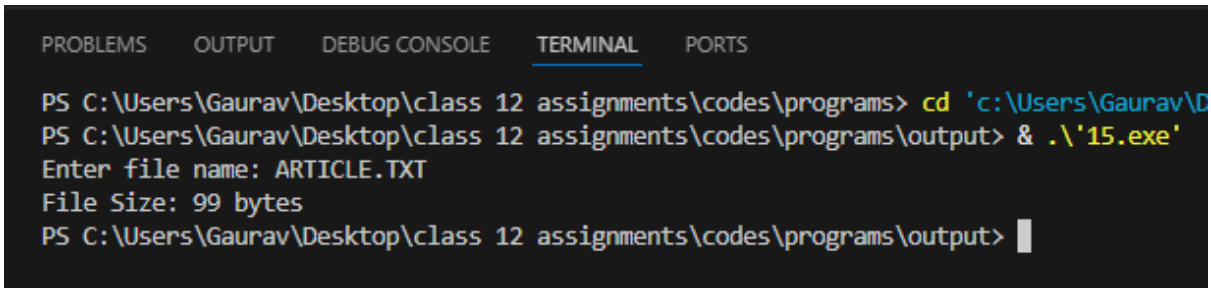


Program 15. : Write a C++ program that displays the size of a file in bytes.

Soln:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
   string filename;
   cout << "Enter file name: ";
   cin >> filename;
   ifstream file(filename, ios::binary | ios::ate);
   if (!file) {
      cout << "Error opening file or file does not exist!\n";
      return 1;
   }
   streampos size = file.tellg();
   file.close();
   cout << "File Size: " << size << " bytes\n";
   return 0;
}
```

Output:

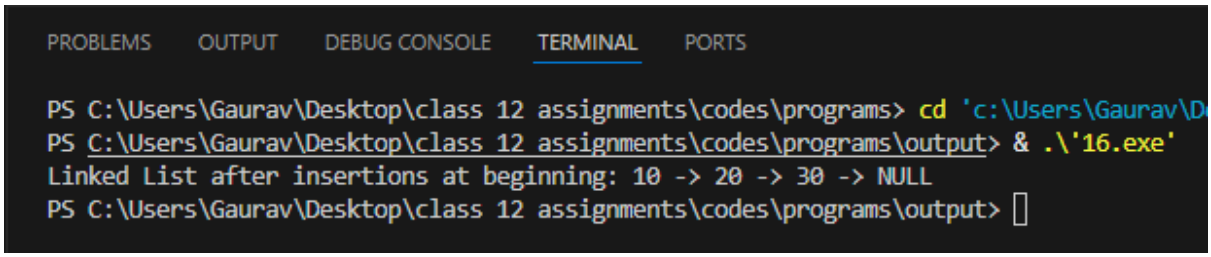


Program 16. : Program for Insertion at the beginning of a List.

Soln:

```
#include <iostream>
using namespace std;
struct Node {
   int data;
   Node* next;
};
void insertAtBeginning(Node*& head, int val) {
   Node* newNode = new Node{val, head};
   head = newNode;
}
void display(Node* head) {
   Node* temp = head;
   while (temp != nullptr) {
      cout << temp->data << " -> ";
      temp = temp->next;
   }
   cout << "NULL\n";
}
int main() {
   Node* head = nullptr;
   insertAtBeginning(head, 30);
   insertAtBeginning(head, 20);
   insertAtBeginning(head, 10);
   cout << "Linked List after insertions at beginning: ";
   display(head);
   return 0;
}
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs> cd 'c:\Users\Gaurav\D
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'16.exe'
Linked List after insertions at beginning: 10 -> 20 -> 30 -> NULL
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output>

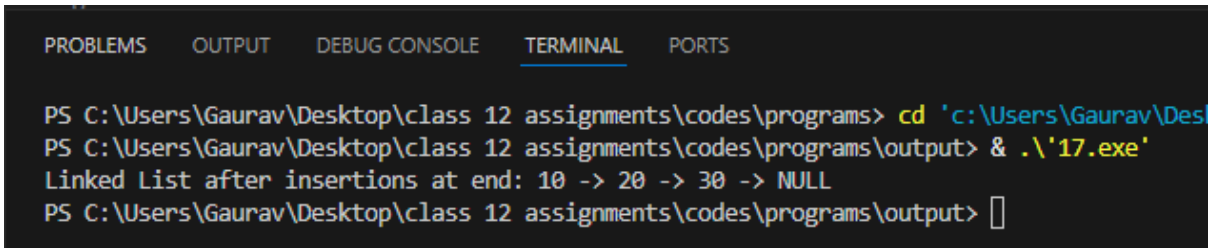
Program 17. : Program for Insertion at the end of a List.

Soln:

```
#include <iostream>
using namespace std;
struct Node {
   int data;
   Node* next;
};
void insertAtEnd(Node*& head, int val) {
   Node* newNode = new Node{val, nullptr};
   if (head == nullptr) {
      head = newNode;
      return;
   }
   Node* temp = head;
   while (temp->next != nullptr) {
      temp = temp->next;
   }
   temp->next = newNode;
}
void display(Node* head) {
   Node* temp = head;
   while (temp != nullptr) {
      cout << temp->data << " -> ";
      temp = temp->next;
   }
   cout << "NULL\n";
}
int main() {
   Node* head = nullptr;
```

```
    insertAtEnd(head, 10);
    insertAtEnd(head, 20);
    insertAtEnd(head, 30);
    cout << "Linked List after insertions at end: ";
    display(head);
    return 0;
}
```

Output:

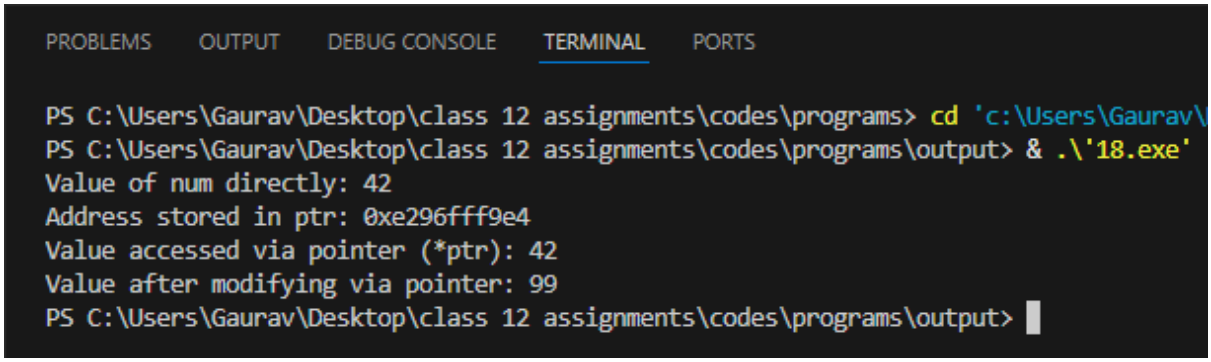


Program 18. : Write a C++ program to declare a pointer to an integer, assign a value to it, and print the value using the pointer.

Soln:

```
#include <iostream>

using namespace std;

int main() {

   int num = 42;

   int* ptr = &num;

   cout << "Value of num directly: " << num << endl;

   cout << "Address stored in ptr: " << ptr << endl;

   cout << "Value accessed via pointer (*ptr): " << *ptr << endl;

   *ptr = 99;

   cout << "Value after modifying via pointer: " << num << endl;

   return 0;

}
```

Output:

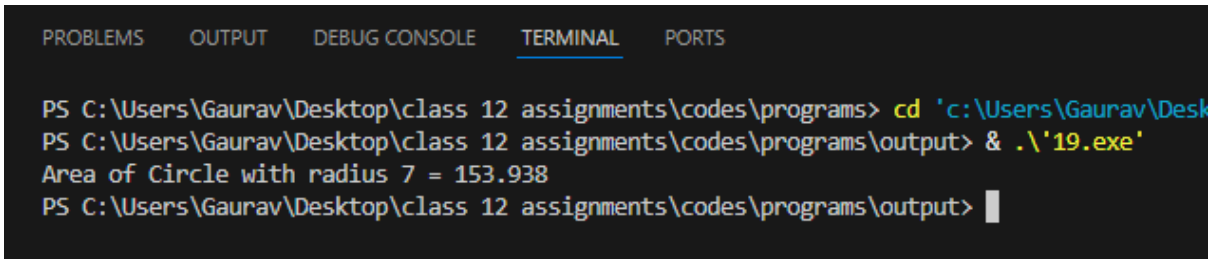


Program 19. : Write a C++ program that demonstrates polymorphism. Create a base class Shape with a function area(). Derive a class Circle and override the area() function to calculate the area of a circle.

Soln:

```
#include <iostream>
using namespace std;
class Shape {
public:
   virtual void area() const {
      cout << "Base Shape area undefined.\n";
   }
   virtual ~Shape() {}
};
class Circle : public Shape {
   double radius;
public:
   Circle(double r) : radius(r) {}
   void area() const override {
      cout << "Area of Circle with radius " << radius
          << " = " << 3.14159 * radius * radius << endl;
   }
};
int main() {
   Shape* shapePtr;
   Circle c(7.0);
   shapePtr = &c;
   shapePtr->area(); // Dynamic dispatch
   return 0;
}
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs> cd 'c:\Users\Gaurav\Desk
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'19.exe'
Area of Circle with radius 7 = 153.938
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output>

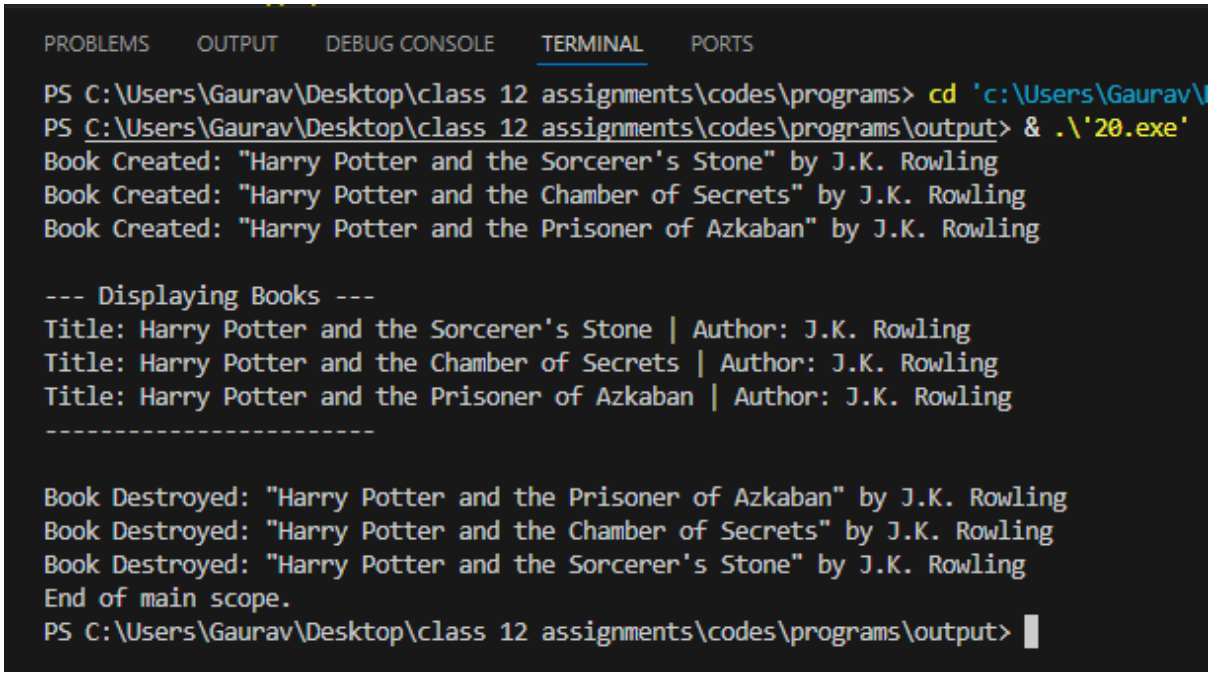
Program 20. : Write a C++ program with a class Book that uses a constructor to initialize title and author, and a destructor that prints a message when an object is destroyed.

Soln:

```
#include <iostream>
using namespace std;
class Book {
   string title;
   string author;
public:
   Book(string t, string a) : title(t), author(a) {
      cout << "Book Created: \"" << title << "\" by " << author << endl;
   }
   ~Book() {
      cout << "Book Destroyed: \"" << title << "\" by " << author << endl;
   }
   void display() const {
      cout << "Title: " << title << " | Author: " << author << endl;
   }
};
int main() {
   {
      Book b1("Harry Potter and the Sorcerer's Stone", "J.K. Rowling");
      Book b2("Harry Potter and the Chamber of Secrets", "J.K. Rowling");
      Book b3("Harry Potter and the Prisoner of Azkaban", "J.K. Rowling");
      cout << "\n--- Displaying Books ---\n";
      b1.display();
      b2.display();
      b3.display();
      cout << "-----------------------\n\n";
   }
```

```
    cout << "End of main scope.\n";
    return 0;
}
```

Output:



Program 21. : Write a C++ program to create a binary file for records (ID, Name, Marks), a text file for logs, update records in the binary file, append data to the text file, and query records by ID with proper error handling using fstream.

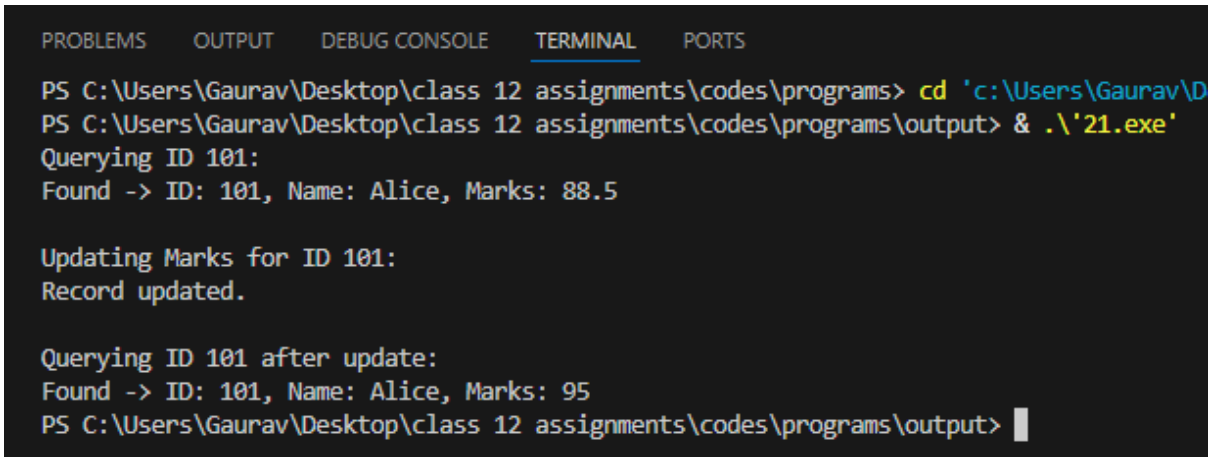
Soln:

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
struct Record {
   int id;
   char name[50];
   float marks;
};
void logMessage(const string& msg) {
   ofstream logFile("log.txt", ios::app);
   if (logFile) {
      logFile << msg << endl;
   }
}
void addRecord(int id, const char* name, float marks) {
   ofstream out("records.dat", ios::binary | ios::app);
   if (!out) {
      cout << "Error opening binary file!\n";
      return;
   }
   Record r;
   r.id = id;
   strncpy(r.name, name, sizeof(r.name) - 1);
   r.name[sizeof(r.name) - 1] = '\0';
   r.marks = marks;
   out.write(reinterpret_cast<char*>(&r), sizeof(Record));
```

```
    out.close();
    logMessage("Added record with ID: " + to_string(id));
}
void queryRecord(int id) {
    ifstream in("records.dat", ios::binary);
    if (!in) {
        cout << "Binary file not found!\n";
        return;
    }
    Record r;
    bool found = false;
    while (in.read(reinterpret_cast<char*>(&r), sizeof(Record))) {
        if (r.id == id) {
       cout << "Found -> ID: " << r.id << ", Name: " << r.name << ", Marks: " << r.marks << endl;
            found = true;
            break;
        }
    }
    if (!found) cout << "Record with ID " << id << " not found.\n";
    in.close();
}
void updateMarks(int id, float newMarks) {
    fstream file("records.dat", ios::binary | ios::in | ios::out);
    if (!file) {
        cout << "File error!\n";
        return;
    }
    Record r;
    bool found = false;
    while (file.read(reinterpret_cast<char*>(&r), sizeof(Record))) {
```

```
        if (r.id == id) {
            r.marks = newMarks;
            file.seekp(-static_cast<streamoff>(sizeof(Record)), ios::cur);
            file.write(reinterpret_cast<char*>(&r), sizeof(Record));
            found = true;
            logMessage("Updated ID " + to_string(id) + " with new marks: " +
to_string(newMarks));
            cout << "Record updated.\n";
            break;
        }
    }
    if (!found) cout << "ID not found for update.\n";
    file.close();
}
int main() {
    addRecord(101, "Alice", 88.5);
    addRecord(102, "Bob", 92.0);
    cout << "Querying ID 101:\n";
    queryRecord(101);
    cout << "\nUpdating Marks for ID 101:\n";
    updateMarks(101, 95.0);
    cout << "\nQuerying ID 101 after update:\n";
    queryRecord(101);
    return 0;
}
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs> cd 'c:\Users\Gaurav\D
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'21.exe'
Querying ID 101:
Found -> ID: 101, Name: Alice, Marks: 88.5
Updating Marks for ID 101:
Record updated.
Querying ID 101 after update:
Found -> ID: 101, Name: Alice, Marks: 95
PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output>

Program 22. : Write a menu-driven C++ program to implement stack operations (push, pop, peek, and display) using a linear stack with arrays, a circular stack with arrays, and a stack with a linked list.

Soln:

```
#include <iostream>

using namespace std;

// 1. Linear Stack using Fixed Array

class LinearStack {

   int top, cap;

   int arr[50];

public:

   LinearStack(int c = 10) : top(-1), cap(c) {}

   void push(int val) {

      if (top == cap - 1) {

         cout << "Stack Overflow!\n";

         return;

      }

      arr[++top] = val;

      cout << "Pushed: " << val << endl;

   }

   void pop() {

      if (top == -1) {

         cout << "Stack Underflow!\n";

         return;

      }

      cout << "Popped: " << arr[top--] << endl;

   }

   void peek() {

      if (top == -1) {

         cout << "Stack is Empty.\n";

      } else {
```

```
            cout << "Top element: " << arr[top] << endl;
        }
    }
    void display() {
        if (top == -1) {
            cout << "Stack is Empty.\n";
            return;
        }
        cout << "Linear Stack (Top to Bottom): ";
        for (int i = top; i >= 0; i--) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};
// 2. Circular Stack using Array (With Wrap-around Logic)
class CircularStack {
    int top, cap, size;
    int arr[50];
public:
    CircularStack(int c = 10) : top(-1), cap(c), size(0) {}
    void push(int val) {
        if (size == cap) {
            cout << "Circular Stack Overflow!\n";
            return;
        }
        top = (top + 1) % cap;
        arr[top] = val;
        size++;
        cout << "Pushed: " << val << endl;
```

```
}
void pop() {
    if (size == 0) {
        cout << "Circular Stack Underflow!\n";
        return;
    }
    cout << "Popped: " << arr[top] << endl;
    top = (top - 1 + cap) % cap;
    size--;
}
void peek() {
    if (size == 0) {
        cout << "Circular Stack is Empty.\n";
    } else {
        cout << "Top element: " << arr[top] << endl;
    }
}
void display() {
    if (size == 0) {
        cout << "Circular Stack is Empty.\n";
        return;
    }
    cout << "Circular Stack (Top to Bottom): ";
    int curr = top;
    for (int i = 0; i < size; i++) {
        cout << arr[curr] << " ";
        curr = (curr - 1 + cap) % cap;
    }
    cout << endl;
}
```

```
};
// 3. Stack using Linked List
struct SNode {
    int data;
    SNode* next;
};
class LinkedStack {
    SNode* top;
public:
    LinkedStack() : top(nullptr) {}
    void push(int val) {
        SNode* newNode = new SNode();
        newNode->data = val;
        newNode->next = top;
        top = newNode;
        cout << "Pushed: " << val << endl;
    }
    void pop() {
        if (!top) {
            cout << "Stack Underflow!\n";
            return;
        }
        SNode* temp = top;
        cout << "Popped: " << temp->data << endl;
        top = top->next;
        delete temp;
    }
    void peek() {
        if (!top) {
            cout << "Stack is Empty.\n";
```

```
        } else {
            cout << "Top element: " << top->data << endl;
        }
    }
    void display() {
        if (!top) {
            cout << "Stack is Empty.\n";
            return;
        }
        cout << "Linked Stack (Top to Bottom): ";
        for (SNode* p = top; p != nullptr; p = p->next) {
            cout << p->data << " -> ";
        }
        cout << "NULL\n";
    }
    // Destructor to clean up memory
    ~LinkedStack() {
        while (top != nullptr) {
            SNode* temp = top;
            top = top->next;
            delete temp;
        }
    }
};
int main() {
    LinearStack ls;
    CircularStack cs;
    LinkedStack lks;
    int type, op, val;
    while (true) {
        cout << "\n==============================\n";
```

```
cout << "         MAIN MENU          \n";
cout << "==============================\n";
cout << "1. Linear Stack (Array)\n";
cout << "2. Circular Stack (Array)\n";
cout << "3. Linked List Stack\n";
cout << "4. Exit Program\n";
cout << "Enter your choice (1-4): ";
cin >> type;
if (type == 4) break;
if (type < 1 || type > 4) {
   cout << "Invalid choice! Please select 1-4.\n";
   continue;
}
while (true) {
   cout << "\n--- Operations Sub-Menu ---\n";
   cout << "1. Push\n2. Pop\n3. Peek\n4. Display\n5. Go back to Main Menu\n";
   cout << "Enter operation (1-5): ";
   cin >> op;
   if (op == 5) break;
   if (op == 1) {
      cout << "Enter value to push: ";
      cin >> val;
   }
   // Route to correct object based on selected main type
   if (type == 1) {
      if (op == 1) ls.push(val);
      else if (op == 2) ls.pop();
      else if (op == 3) ls.peek();
      else if (op == 4) ls.display();
      else cout << "Invalid operation!\n";
```

```
        }
        else if (type == 2) {
            if (op == 1) cs.push(val);
            else if (op == 2) cs.pop();
            else if (op == 3) cs.peek();
            else if (op == 4) cs.display();
            else cout << "Invalid operation!\n";
        }
        else if (type == 3) {
            if (op == 1) lks.push(val);
            else if (op == 2) lks.pop();
            else if (op == 3) lks.peek();
            else if (op == 4) lks.display();
            else cout << "Invalid operation!\n";
        }
      }
   }
   cout << "\nProgram exited successfully.\n";
   return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Gaurav\Desktop\class 12 assignments\codes\programs\output> & .\'22.exe'

==============================
         MAIN MENU
==============================
1. Linear Stack (Array)
2. Circular Stack (Array)
3. Linked List Stack
4. Exit Program
Enter your choice (1-4): 3

--- Operations Sub-Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Go back to Main Menu
Enter operation (1-5): 1
Enter value to push: 50
Pushed: 50

--- Operations Sub-Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Go back to Main Menu
Enter operation (1-5): 1
Enter value to push: 100
Pushed: 100

--- Operations Sub-Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Go back to Main Menu
Enter operation (1-5): 4
Linked Stack (Top to Bottom): 100 -> 50 -> NULL
```

Assignment: SQL Commands with Usage and Output

1. Creating a Table :

This command is used to create a table named Employees with columns ID, Name, Role, Salary, Joining_Date, and Company. Each column is defined with its data type and constraints.

```
mysql> USE PARADOX;
Database changed
mysql> CREATE TABLE Employees (
    -> ID INT PRIMARY KEY,
    -> NAME VARCHAR(50),
    -> ROLE VARCHAR(50),
    -> SALARY INT,
    -> JOINING_DATE DATE,
    -> COMPANY VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
```

2. Insert Records :

This command inserts new records into the Employees table with specific values for each column.

```
mysql> INSERT INTO Employees (ID, Name, Role, Salary, Joining_Date, Company) VALUES
    -> (1, 'Rahul Sharma', 'Developer', 50000, '2024-11-01', 'Skillarger'),
    -> (2, 'Anjali Mehta', 'Developer', 45000, '2024-10-15', 'Skillarger'),
    -> (3, 'Amitabh Sinha', 'Manager', 70000, '2024-09-20', 'Skillarger'),
    -> (4, 'Priya Singh', 'HR', 40000, '2024-07-10', 'Skillarger'),
    -> (5, 'Ravi Patel', 'Analyst', 55000, '2024-08-05', 'Skillarger');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

3. Select All Records :

This command retrieves all rows and columns from the Employees table, displaying the complete dataset.

```
mysql> SELECT * FROM Employees;
+----+--------------+----------+--------+--------------+------------+
| ID | NAME         | ROLE     | SALARY | JOINING_DATE | COMPANY    |
+----+--------------+----------+--------+--------------+------------+
|  1 | Rahul Sharma | Developer |  50000 | 2024-11-01   | Skillarger |
|  2 | Anjali Mehta | Developer |  45000 | 2024-10-15   | Skillarger |
|  3 | Amitabh Sinha | Manager  |  70000 | 2024-09-20   | Skillarger |
|  4 | Priya Singh  | HR       |  40000 | 2024-07-10   | Skillarger |
|  5 | Ravi Patel   | Analyst  |  55000 | 2024-08-05   | Skillarger |
+----+--------------+----------+--------+--------------+------------+
5 rows in set (0.00 sec)
```

4. Select Specific Columns :

This command retrieves only the Name and Role columns from all employees, ignoring the rest of the data.

```
mysql> SELECT Name, Role FROM Employees;
+---------------+-----------+
| Name          | Role      |
+---------------+-----------+
| Rahul Sharma  | Developer |
| Anjali Mehta  | Developer |
| Amitabh Sinha | Manager   |
| Priya Singh   | HR        |
| Ravi Patel    | Analyst   |
+---------------+-----------+
5 rows in set (0.00 sec)
```

5. Filter by Salary :

This command retrieves records of employees whose salaries are greater than 50,000.

```
mysql> SELECT * FROM Employees WHERE Salary > 50000;
+----+---------------+---------+--------+--------------+------------+
| ID | NAME          | ROLE    | SALARY | JOINING_DATE | COMPANY    |
+----+---------------+---------+--------+--------------+------------+
|  3 | Amitabh Sinha | Manager |  70000 | 2024-09-20   | Skillarger |
|  5 | Ravi Patel    | Analyst |  55000 | 2024-08-05   | Skillarger |
+----+---------------+---------+--------+--------------+------------+
2 rows in set (0.00 sec)
```

6. Update a Record :

This command updates the salary of the employee with ID = 2 to 60,000.

```
mysql> UPDATE Employees SET Salary = 60000 WHERE ID = 2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

7. Delete a Record :

This command deletes the record of the employee with ID = 4.

```
mysql> DELETE FROM Employees WHERE ID = 4;
Query OK, 1 row affected (0.01 sec)
```

8. Count Total Records :

This command counts the total number of rows (records) in the Employees table.

```
mysql> SELECT COUNT(*) AS Total_Employees FROM Employees;
+-----------------+
| Total_Employees |
+-----------------+
|               4 |
+-----------------+
1 row in set (0.01 sec)
```

9. Sort Records :

This command sorts the rows of the Employees table in descending order based on the Salary column.

```
mysql> SELECT * FROM Employees ORDER BY Salary DESC;
+----+--------------+----------+--------+--------------+------------+
| ID | NAME         | ROLE     | SALARY | JOINING_DATE | COMPANY    |
+----+--------------+----------+--------+--------------+------------+
|  3 | Amitabh Sinha | Manager  |  70000 | 2024-09-20   | Skillarger |
|  2 | Anjali Mehta  | Developer |  60000 | 2024-10-15   | Skillarger |
|  5 | Ravi Patel    | Analyst  |  55000 | 2024-08-05   | Skillarger |
|  1 | Rahul Sharma  | Developer |  50000 | 2024-11-01   | Skillarger |
+----+--------------+----------+--------+--------------+------------+
4 rows in set (0.00 sec)
```

10. Find Maximum Salary :

This command retrieves the maximum salary value from the Employees table.

```
mysql> SELECT MAX(Salary) AS Highest_Salary FROM Employees;
+----------------+
| Highest_Salary |
+----------------+
|          70000 |
+----------------+
1 row in set (0.00 sec)
```

11. Group by Role :

This command groups the data by the Role column and counts the number of employees in each role.

```
mysql> SELECT Role, COUNT(*) AS Total FROM Employees GROUP BY Role;
+-----------+-------+
| Role      | Total |
+-----------+-------+
| Developer |     2 |
| Manager   |     1 |
| Analyst   |     1 |
+-----------+-------+
3 rows in set (0.00 sec)
```

12. Drop the Table :

This command deletes the Employees table and all its data permanently from the database.

```
mysql> DROP TABLE XYZ;
Query OK, 0 rows affected (0.02 sec)
```

13. Add a New Column :

This command adds a new column Email to the Employees table.

```
mysql> ALTER TABLE Employees ADD Email VARCHAR(100);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

14. Update Email Addresses :

This command generates email addresses for employees by combining their names and the company domain.

```
mysql> UPDATE Employees SET Email = 'rahul@gmail.com' WHERE ID = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Employees SET Email = 'anjali@gmail.com' WHERE ID = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Employees SET Email = 'amitabh@gmail.com' WHERE ID = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Employees SET Email = 'priya@gmail.com' WHERE ID = 4;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> UPDATE Employees SET Email = 'ravi@gmail.com' WHERE ID = 5;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

15. Select Records with Email Column :

This command retrieves the ID, Name, and Email columns from the Employees table.

```
mysql> SELECT ID, Name, Email FROM Employees;
+----+--------------+------------------+
| ID | Name         | Email            |
+----+--------------+------------------+
|  1 | Rahul Sharma | rahul@gmail.com  |
|  2 | Anjali Mehta | anjali@gmail.com |
|  3 | Amitabh Sinha| amitabh@gmail.com|
|  5 | Ravi Patel   | ravi@gmail.com   |
+----+--------------+------------------+
4 rows in set (0.00 sec)
```

16. Filter by Joining Date :

This command retrieves records of employees who joined after September 1, 2024.

```
mysql> SELECT * FROM Employees WHERE Joining_Date > '2024-09-01';
+----+---------------+-----------+--------+--------------+------------+-------------------+
| ID | NAME          | ROLE      | SALARY | JOINING_DATE | COMPANY    | Email             |
+----+---------------+-----------+--------+--------------+------------+-------------------+
|  1 | Rahul Sharma  | Developer |  50000 | 2024-11-01   | Skillarger | rahul@gmail.com   |
|  2 | Anjali Mehta  | Developer |  60000 | 2024-10-15   | Skillarger | anjali@gmail.com  |
|  3 | Amitabh Sinha | Manager   |  70000 | 2024-09-20   | Skillarger | amitabh@gmail.com |
+----+---------------+-----------+--------+--------------+------------+-------------------+
3 rows in set (0.00 sec)
```

17. Rename Column :

This command renames the column Email to Contact_Email in the Employees table.

```
mysql> ALTER TABLE Employees RENAME COLUMN Email TO Contact_Email;
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

18. Add a New Employee :

This command inserts a new employee record into the Employees table.

```
mysql> INSERT INTO Employees (ID, Name, Role, Salary, Joining_Date, Company, Contact_Email) VALUES (6, 'Neha Verma', 'Tester', 48000, '2024-12-01',
'Skillarger', 'neha@gmail.com');
Query OK, 1 row affected (0.01 sec)
```

19. Show Updated Table :

This command displays all records from the updated Employees table, including the newly added employee.

```
mysql> SELECT * FROM Employees;
+----+---------------+-----------+--------+--------------+------------+-------------------+
| ID | NAME          | ROLE      | SALARY | JOINING_DATE | COMPANY    | Contact_Email     |
+----+---------------+-----------+--------+--------------+------------+-------------------+
|  1 | Rahul Sharma  | Developer |  50000 | 2024-11-01   | Skillarger | rahul@gmail.com   |
|  2 | Anjali Mehta  | Developer |  60000 | 2024-10-15   | Skillarger | anjali@gmail.com  |
|  3 | Amitabh Sinha | Manager   |  70000 | 2024-09-20   | Skillarger | amitabh@gmail.com |
|  5 | Ravi Patel    | Analyst   |  55000 | 2024-08-05   | Skillarger | ravi@gmail.com    |
|  6 | Neha Verma    | Tester    |  48000 | 2024-12-01   | Skillarger | neha@gmail.com    |
+----+---------------+-----------+--------+--------------+------------+-------------------+
5 rows in set (0.00 sec)
```

20. Create a View :

This command creates a view named HighEarners to display employees whose salary is greater than 50,000.

```
mysql> CREATE VIEW HighEarners AS SELECT * FROM Employees WHERE Salary > 50000;
Query OK, 0 rows affected (0.01 sec)
```

21. Display View Data :

This command retrieves all records from the HighEarners view.

```
mysql> SELECT * FROM HighEarners;
+----+---------------+-----------+--------+--------------+------------+-------------------+
| ID | NAME          | ROLE      | SALARY | JOINING_DATE | COMPANY    | Contact_Email     |
+----+---------------+-----------+--------+--------------+------------+-------------------+
|  2 | Anjali Mehta  | Developer |  60000 | 2024-10-15   | Skillarger | anjali@gmail.com  |
|  3 | Amitabh Sinha | Manager   |  70000 | 2024-09-20   | Skillarger | amitabh@gmail.com |
|  5 | Ravi Patel    | Analyst   |  55000 | 2024-08-05   | Skillarger | ravi@gmail.com    |
+----+---------------+-----------+--------+--------------+------------+-------------------+
3 rows in set (0.00 sec)
```

21 SQL commands we used on this project

1. **Create Table :** CREATE TABLE Employees (...);
2. **Insert Records :** INSERT INTO Employees (...) VALUES (...);
3. **Select All Records :** SELECT * FROM Employees;
4. **Select Specific Columns :** SELECT Name, Role FROM Employees;
5. **Filter by Salary :** SELECT * FROM Employees WHERE Salary > 50000;
6. **Update a Record :** UPDATE Employees SET Salary = 60000 WHERE ID = 2;
7. **Delete a Record :** DELETE FROM Employees WHERE ID = 4;
8. **Count Total Records :** SELECT COUNT(*) FROM Employees;
9. **Sort Records :** SELECT * FROM Employees ORDER BY Salary DESC;
10. **Find Maximum Salary :** SELECT MAX(Salary) AS Highest_Salary FROM Employees;
11. **Group by Role :** SELECT Role, COUNT(*) AS Total FROM Employees GROUP BY Role;
12. **Drop Table :** DROP TABLE Employees;
13. **Add a New Column :** ALTER TABLE Employees ADD Email VARCHAR(100);
14. **Update Email addresses :** UPDATE Employees SET Email = CONCAT(LOWER(Name), '@skillarger.com');
15. **Select Record with Email Column :** SELECT ID, Name, Email FROM Employees;
16. **Filter by joining data :** SELECT * FROM Employees WHERE Joining_Date > '2024-09-01';
17. **Rename Column :** ALTER TABLE Employees RENAME COLUMN Email TO Contact_Email;
18. **Add new Employee :** INSERT INTO Employees (...) VALUES (...);
19. **Show Updated Table :** SELECT * FROM Employees;
20. **Create a View :** CREATE VIEW HighEarners AS SELECT ...;
21. **Display View Date :** SELECT * FROM HighEarners;